

Task 2

Example 1 — Read JSON as a Stream

First, let's only learn how to **read JSON continuously**.

Assume your JSON files are here:

```
hdfs://localhost:9000/data/json/
```

For streaming, Spark watches a **directory**, not normally one fixed JSON file.

Step 1: Import Spark

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName("JSON Streaming")
        .getOrCreate()
)

print("Spark version:", spark.version)
```

Step 2: Define the schema

For Structured Streaming, it is a good practice to explicitly define the schema.

For example, suppose your JSON contains:

```
{
  "order_id": "ORD1001",
  "timestamp": "2026-08-27T10:15:00Z",
  "customer": {
    "customer_id": 501,
    "name": "John Doe",
    "email": "john@example.com",
    "address": {
      "city": "Toronto"
    }
  },
  "product": "Laptop",
  "category": "Electronics",
  "price": 1200.0,
  "quantity": 1
}
```

We create the schema:

```
from pyspark.sql.types import *

schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("timestamp", StringType(), True),

    StructField("customer", StructType([
        StructField("customer_id", IntegerType(), True),
        StructField("name", StringType(), True),
        StructField("email", StringType(), True),

        StructField("address", StructType([
            StructField("city", StringType(), True)
        ]), True),

    ]), True),

    StructField("product", StringType(), True),
    StructField("category", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("quantity", IntegerType(), True)
])
```

Step 3: Read JSON as streaming

This is the important part:

```
df = (
    spark.readStream
        .schema(schema)
        .json("hdfs://localhost:9000/data/json/")
)
```

Notice:

```
spark.readStream
```

instead of:

```
spark.read
```

Batch

```
df = spark.read.json(...)
```

means:

Read the files that exist now.

Streaming

```
df = spark.readStream.json(...)
```

means:

Keep watching the directory for new files.

Step 4: Look at the schema

```
df.printSchema()
```

You should see something similar to:

```
root
 |-- order_id: string
 |-- timestamp: string
 |-- customer: struct
 |   |-- customer_id: integer
 |   |-- name: string
 |   |-- email: string
 |   |-- address: struct
 |       |-- city: string
 |-- product: string
 |-- category: string
 |-- price: double
 |-- quantity: integer
```

Step 5: We need a sink

A streaming DataFrame doesn't actually start processing just because we created it.

This:

```
df = spark.readStream...
```

only creates the streaming DataFrame.

We need:

```
Source
  ↓
Transformation
  ↓
Sink
```

For our first example, let's display the data in the terminal.

```
query = (  
    df.writeStream  
        .format("console")  
        .outputMode("append")  
        .option("truncate", False)  
        .start()  
)
```

Then:

```
query.awaitTermination()
```

Complete Example 1

```
from pyspark.sql import SparkSession  
from pyspark.sql.types import *  
  
# =====  
# 1. Create Spark Session  
# =====  
  
spark = (  
    SparkSession.builder  
        .appName("JSON Streaming")  
        .getOrCreate()  
)  
  
print("Spark version:", spark.version)  
  
# =====  
# 2. Define Schema  
# =====  
  
schema = StructType([  
    StructField("order_id", StringType(), True),  
    StructField("timestamp", StringType(), True),  
  
    StructField("customer", StructType([  
        StructField("customer_id", IntegerType(), True),  
        StructField("name", StringType(), True),  
        StructField("email", StringType(), True),  
  
        StructField("address", StructType([  
            StructField("city", StringType(), True)  
        ]), True),  
    ]), True),  
  
    StructField("product", StringType(), True),
```

```

        StructField("category", StringType(), True),
        StructField("price", DoubleType(), True),
        StructField("quantity", IntegerType(), True)
    ])

# =====
# 3. Read JSON as Streaming
# =====

df = (
    spark.readStream
        .schema(schema)
        .json("hdfs://localhost:9000/data/json/")
)

# =====
# 4. Show Schema
# =====

df.printSchema()

# =====
# 5. Write Stream to Console
# =====

query = (
    df.writeStream
        .format("console")
        .outputMode("append")
        .option("truncate", False)
        .start()
)

# =====
# 6. Keep Streaming Application Running
# =====

query.awaitTermination()

```

The architecture you should remember

For now, memorize this:

```

JSON files
  ↓
readStream
  ↓
Streaming DataFrame
  ↓
transformations

```

```
↓  
writeStream  
↓  
sink
```

For example:

```
HDFS JSON  
↓  
spark.readStream  
↓  
df  
↓  
select / filter / withColumn  
↓  
writeStream  
↓  
Console / JSON / Parquet
```

Example 2 — Transform the JSON

Now let's actually transform the nested JSON.

We can extract:

```
customer.customer_id  
customer.name  
customer.address.city
```

and create:

```
total_price = price * quantity
```

Use:

```
from pyspark.sql.functions import *
```

Then:

```
transformed_df = df.select(  
    col("order_id"),  
    col("timestamp"),  
    col("customer.customer_id").alias("customer_id"),  
    col("customer.name").alias("customer_name"),  
    col("customer.address.city").alias("city"),  
    col("product"),  
    col("category"),  
    col("price"),  
    col("quantity"),  
    (col("price") * col("quantity")).alias("total_price")
```

```
)
```

Now your streaming data becomes much flatter:

```
order_id  
timestamp  
customer_id  
customer_name  
city  
product  
category  
price  
quantity  
total_price
```

Then write it to the console:

```
query = (  
    transformed_df.writeStream  
        .format("console")  
        .outputMode("append")  
        .option("truncate", False)  
        .start()  
)  
  
query.awaitTermination()
```

So now the pipeline is:

```
JSON  
↓  
readStream  
↓  
df  
↓  
select()  
↓  
transformed_df  
↓  
writeStream  
↓  
console
```